

Análisis de expresión génica y enriquecimiento funcional en un modelo murino de infección por *Staphylococcus aureus* bajo tratamiento con Linezolid y Vancomicina

Daniel Camacho Montaña

November 2025

Resumen

En este trabajo se investigó el efecto de la infección por *Staphylococcus aureus* y de los tratamientos con antibióticos linezolid y vancomicina en la expresión génica de un modelo murino. Se seleccionaron 24 muestras representativas de los grupos experimentales, se procesaron los datos crudos de microarrays Affymetrix y se construyó un objeto **ExpressionSet**. Tras normalizar los datos mediante RMA y realizar controles de calidad, se identificaron genes diferencialmente expresados mediante modelos lineales ajustados con **limma**. Posteriormente, se aplicó un análisis de enriquecimiento funcional de Gene Ontology (GO) para evaluar los procesos biológicos implicados en cada contraste experimental. Los resultados muestran que la infección induce respuestas inmunes y metabólicas específicas, mientras que los tratamientos antibióticos generan efectos distintos sobre el transcriptoma, evidenciando la modulación particular de procesos biológicos según el agente utilizado. Este enfoque integral permite caracterizar los cambios funcionales asociados a la infección y a la terapia antibiótica, proporcionando información relevante para la interpretación biológica de los datos transcriptómicos.

Índice

1. Introducción y objetivos	3
1.1. Preparación de los archivos CEL	4
2. Métodos	5
2.1. Creación del ExpressionSet	5
2.2. Análisis exploratorio y control de calidad	6
2.2.1. PCA: Análisis de Componentes Principales	9
2.3. Filtro de los Datos	12
2.4. Construcción de la Matriz de Diseño y de Contraste	13
2.5. Obtención de los Genes Diferencialmente Expresados	14
2.6. Anotación de los Genes	15
2.7. Análisis de la Significación Biológica	17
3. Resultados	19

1. Introducción y objetivos

Se utilizó un modelo de ratón para investigar la utilidad de los antibióticos LINEZOLID y VANCOMICINA en la inmunomodulación durante infecciones por *Staphylococcus aureus* resistente a meticilina. La información sobre las muestras y sus condiciones experimentales se encuentra en el fichero `allTargets.txt`.

El dataset contiene datos de 35 muestras: 15 tomadas antes de la infección, 20 después, 5 correspondientes a las 2 horas que serán eliminadas y 15 a las 24 horas. El objetivo consiste en caracterizar, a través del cambio en la expresión génica, el efecto de la infección y del tratamiento con antibióticos, así como comparar los efectos de ambos tratamientos.

Se plantearon las siguientes comparaciones:

- Infectados vs. no infectados sin tratamiento
- Infectados vs. no infectados tratados con LINEZOLID
- Infectados vs. no infectados tratados con VANCOMICINA

Para la obtención de los datos desde Gene Expression Omnibus (GEO), se cargaron los paquetes BioConductor y GEOquery. Los datos del experimento, identificados como **GSE38531**, se descargaron mediante:

```
getGEOSuppFiles("GSE38531", makeDirectory = TRUE)
```

Esto generó un archivo comprimido `GSE38531_RAW.tar` con los datos crudos. La descompresión se realizó con la función `untar`, especificando el directorio creado para la descarga:

```
untar("GSE38531/GSE38531_RAW.tar", exdir = "GSE38531_RAW")
```

Los archivos descomprimidos corresponden a cada muestra, identificados con el mismo nombre que figura en la columna `sample` de `allTargets`, y se almacenan en la carpeta `GSE38531_RAW`.

Para simplificar el análisis y reducir el riesgo de intercambio no permitido de información, se eliminaron las muestras correspondientes a las 2 horas (5 muestras) y se seleccionaron aleatoriamente 4 muestras de cada grupo, obteniendo un total de 12 muestras para 3 grupos, o 24 si se considera la selección por interacción de variables (6 grupos experimentales).

Para realizar el filtrado y el muestreo se definió la función `filter_microarray`:

```
filter_microarray <- function(allTargets, seed = 39432449) {
  # Configuración de la semilla para reproducibilidad
  set.seed(seed)

  # Filtrado de filas donde 'time' no sea 'hour 2'
  filtered <- subset(allTargets, time != "hour 2")

  # Creación de la columna 'group' como interacción entre 'infection' y 'agent'
  filtered$group <- interaction(filtered$infection, filtered$agent)
```

```

# Selección aleatoria de 4 muestras por grupo
selected <- do.call(rbind, lapply(split(filtered, filtered$group),
  function(group_data) {
    if (nrow(group_data) > 4) {
      group_data[sample(1:nrow(group_data), 4), ]
    } else {
      group_data
    }
  })

# Obtención de los índices originales de las muestras seleccionadas
original_indices <- match(selected$sample, allTargets$sample)

# Asignación de los índices a los nombres de filas
rownames(selected) <- paste0(selected$sample, ".", original_indices)

# Eliminación de la columna 'group' y devolución del resultado
selected$group <- NULL
return(selected)
}

```

La función se aplicó al conjunto `allTargets`, generando un `data.frame` con 24 muestras seleccionadas, cuyas filas se nombraron según los índices originales del dataset. Las variables categóricas (`infection`, `time`, `agent`) se convirtieron en factores para su posterior análisis.

Esta metodología garantiza que el conjunto de datos utilizado para los análisis contiene una representación balanceada de los grupos experimentales, facilitando comparaciones robustas de los efectos de la infección y de los tratamientos antibióticos.

1.1. Preparación de los archivos CEL

Los archivos `.CEL` de las muestras contienen los datos crudos obtenidos directamente de los experimentos de microarrays Affymetrix. Para generar un `ExpressionSet`, se descomprimieron previamente los archivos `.gz` ubicados en `GSE38531_RAW` mediante `lapply`:

```

gz_files <- list.files(path = "GSE38531_RAW", pattern = "\\*.gz$",
  full.names = TRUE)
invisible(lapply(gz_files, function(x) gunzip(x, remove = FALSE)))

```

Una vez descomprimidos, se almacenaron todos los ficheros CEL en una lista denominada `CEL_files`. Dado que los nombres de los archivos originales presentan extensiones largas y heterogéneas, se renombraron para mantener únicamente el identificador de la muestra y se generó la lista `CEL_files` para reflejar los cambios:

```

CEL_files <- list.files(path = "GSE38531_RAW", pattern = "\\*.CEL$",
  full.names = TRUE)
newNames <- paste0(substr(basename(CEL_files), 1, 9), ".CEL")
file.rename(CEL_files, file.path(dirname(CEL_files), newNames))
CEL_files <- list.files(path = "GSE38531_RAW", pattern = "\\*.CEL$",
  full.names = TRUE)

```

Finalmente, se filtraron los archivos CEL para conservar únicamente aquellos correspondientes a las 24 muestras seleccionadas, eliminando los restantes del directorio:

```
celFileNames <- substr(basename(CEL_files), 1, nchar(basename(CEL_files)) - 4)
filesToDelete <- CEL_files[!(celFileNames %in% Targets$sample)]
file.remove(filesToDelete)
```

La función `file.remove` devuelve `TRUE` para cada archivo eliminado, confirmando la eliminación efectiva. El resumen de las muestras seleccionadas es el siguiente:

```
sample           infection      time      agent
Length:24        S. aureus USA300:12  hour 0 :12  linezolid :8
Class :character  uninfected          :12  hour 24:12  untreated :8
Mode  :character                                     vancomycin:8
```

Este procedimiento asegura que el conjunto de datos final contiene únicamente las muestras seleccionadas, debidamente renombradas y categorizadas, permitiendo su uso en análisis posteriores de expresión génica.

2. Métodos

2.1. Creación del ExpressionSet

Una vez determinadas las muestras que formarán parte del `ExpressionSet`, se almacenaron en un objeto denominado `rawData`. Para ello, se especificó el directorio de trabajo y se definieron las rutas para los resultados y los datos de origen:

```
workingDir <- getwd()
dataDir <- file.path(workingDir, "GSE38531_RAW")
resultsDir <- file.path(workingDir, "results")
```

Se cargaron los paquetes `affy` y `Biobase` para permitir la lectura de los archivos CEL y la creación del `ExpressionSet`.

Los nombres de las muestras se asignaron a partir de la variable `sample` del `data.frame` `Targets`. Posteriormente, se creó un objeto `AnnotatedDataFrame` con la información de las muestras filtradas, incluyendo los metadatos del grupo experimental:

```
row.names(Targets) <- Targets$sample
Targets_Annotated <- AnnotatedDataFrame(Targets)
```

El objeto resultante es un `AnnotatedDataFrame` con 24 observaciones y 4 variables (`sample`, `infection`, `time` y `agent`), que contiene la información necesaria para asociar cada muestra con su grupo experimental correspondiente.

Los archivos CEL del directorio `GSE38531_RAW`, previamente filtrados mediante la función `filter_microarray`, se cargaron en un objeto `AffyBatch` mediante la función `ReadAffy`, utilizando `Targets_Annotated` para especificar los metadatos:

```
CEL_files <- list.files(path = dataDir, pattern = "\\\\.CEL$", full.names = TRUE)
rawData <- ReadAffy(filename = CEL_files, phenoData = Targets_Annotated)
```

A partir del objeto `AffyBatch`, se extrajeron los datos de expresión (`exprs`), la información de las muestras (`pData`) y la información de los genes (`fData`):

```
exprs_data <- exprs(rawData)
pheno_data <- pData(rawData)
feature_data <- fData(rawData)
```

Con esta información se construyó el `ExpressionSet`:

```
expression_set <- ExpressionSet(assayData = exprs_data,
                                phenoData = AnnotatedDataFrame(pheno_data),
                                featureData = AnnotatedDataFrame(feature_data))
```

El objeto resultante es un `ExpressionSet` que contiene los datos de expresión, los metadatos de las muestras y la información de los genes, listo para su uso en análisis posteriores. La estructura del `ExpressionSet` incluye los slots `experimentData`, `assayData`, `phenoData`, `featureData`, `annotation` y `protocolData`, garantizando que todos los datos y metadatos estén correctamente asociados y etiquetados.

2.2. Análisis exploratorio y control de calidad

Se realizó un análisis exploratorio sobre el `ExpressionSet` para evaluar la distribución de los datos. Los datos de expresión se extrajeron mediante la función `exprs()` y se construyó un `boxplot` para visualizar las intensidades de señal de cada muestra:

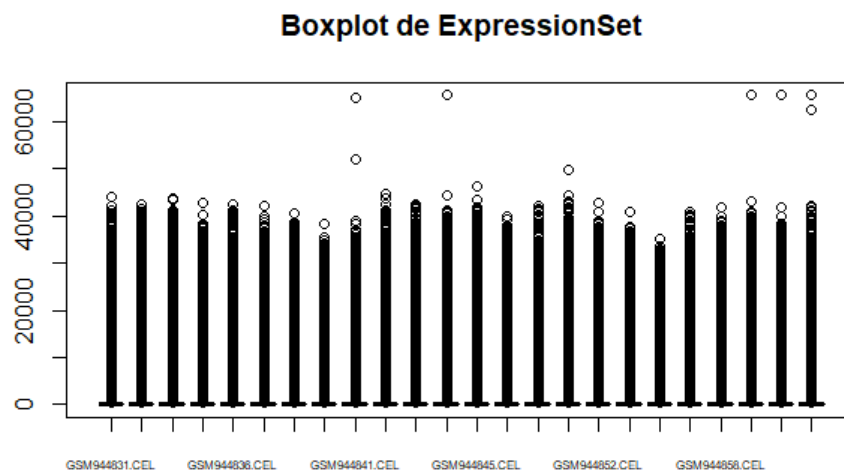


Figura 1: Distribución de las intensidades de expresión de todas las muestras del `ExpressionSet`.

El `boxplot` indica que las cajas presentan alturas y posiciones similares entre todas las muestras, sugiriendo que las intensidades crudas no presentan diferencias globales significativas. La presencia de valores atípicos es habitual en datos crudos de microarrays. Para corregir diferencias sistemáticas, se aplicó la normalización RMA (Robust Multi-Array Average), que incluye corrección de *background*, normalización y *summarization*:

```
normalized_data <- rma(rawData)
```

Posteriormente, se generó un `boxplot` con los datos normalizados, mostrando una distribución homogénea y confirmando la efectividad de la normalización:

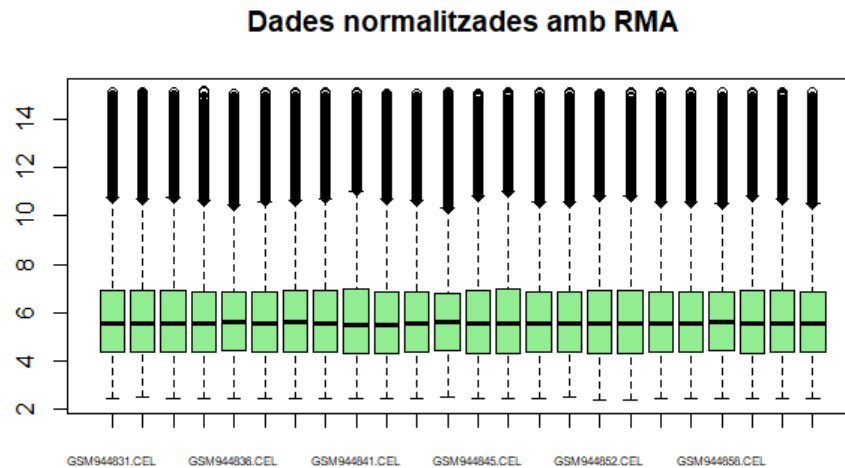


Figura 2: Distribución de las intensidades de expresión de las muestras tras normalización RMA.

Se evaluó la calidad de los datos utilizando el paquete `arrayQualityMetrics`, aplicando el control sobre los datos normalizados y crudos para comparar la mejora tras la normalización:

```
library(arrayQualityMetrics)
arrayQualityMetrics(expressionset = normalized_data,
  outdir = file.path(resultsDir, "control_calidad"),
  force = TRUE)
```

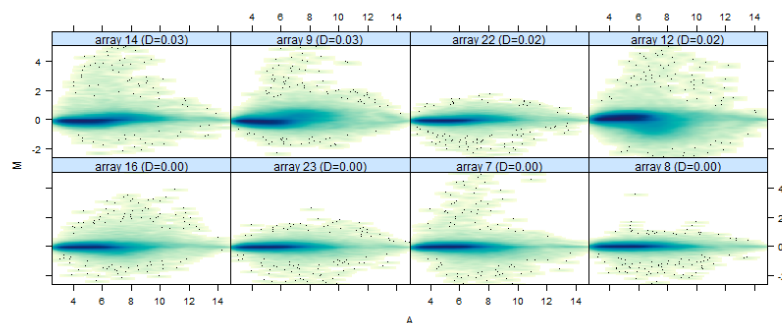


Figura 3: MA Plot: Diferencias relativas (M) frente a intensidad promedio (A).

En este gráfico, los puntos representan las sondas del microarray. Idealmente, los puntos deberían distribuirse simétricamente alrededor de la línea $M = 0$. Se observa que la mayoría de los puntos se centra en el eje central, aunque algunos se alejan ligeramente.

No se detectan patrones sistemáticos ni estructuras anómalas significativas, lo que indica que los datos normalizados presentan buena calidad.

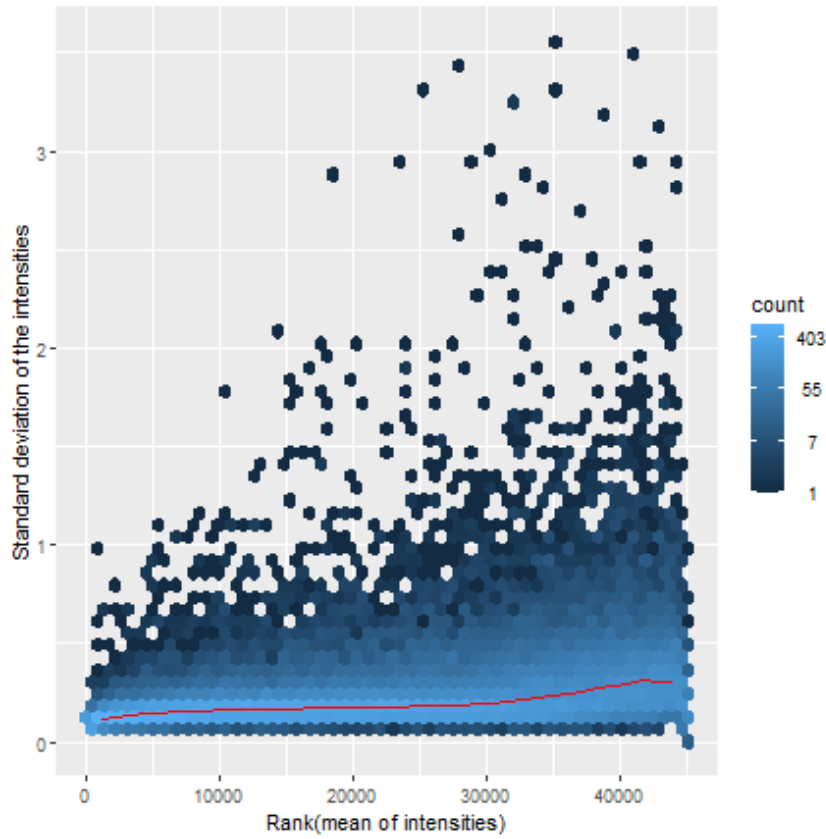


Figura 4: MSD Plot: Relación entre intensidad promedio y desviación estándar.

El MSD Plot permite evaluar cómo varía la desviación estándar en función de la intensidad promedio. La mayoría de los puntos se encuentra concentrada cerca de la base, con algunos valores más dispersos, pero sin variaciones excesivas. La curva roja representa la tendencia general de la desviación estándar. La ligera inclinación observada indica que los datos normalizados mantienen una calidad consistente.

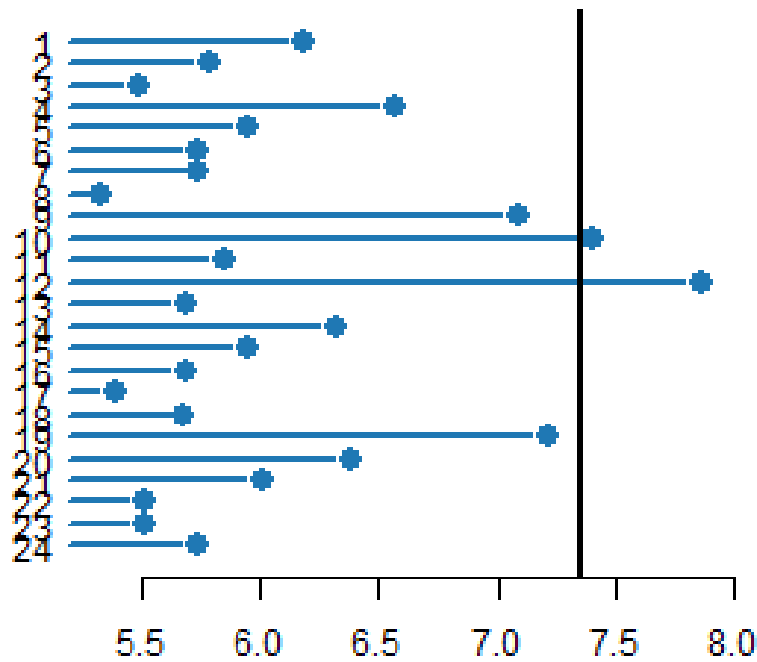


Figura 5: Heatmap de correlación entre muestras (OUT_HM).

Este heatmap permite evaluar la similitud entre las muestras. Se observa que dos muestras se alejan del agrupamiento principal, lo que indica que presentan patrones de expresión distintos respecto al resto. Esta desviación puede deberse a diferencias biológicas marcadas, baja calidad de la muestra o errores puntuales en la normalización.

2.2.1. PCA: Análisis de Componentes Principales

Se realizó un análisis de componentes principales (PCA) para reducir la dimensionalidad de los datos y evaluar posibles agrupamientos por factores experimentales:

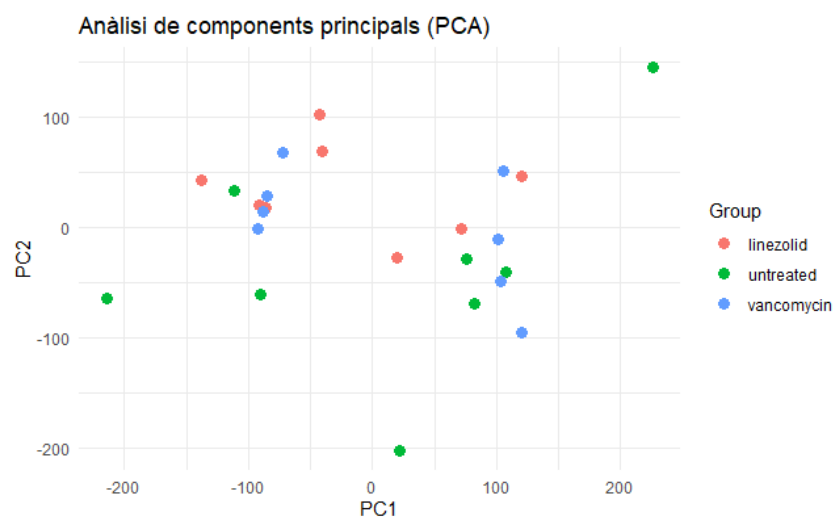


Figura 6: Gráfico PCA de los datos normalizados. Cada punto representa una muestra, coloreada según el grupo (agent).

No se aprecia un agrupamiento en función del tratamiento (**agent**), aunque se observa una posible separación en dos grupos no relacionados, lo que podría indicar un efecto *batch* sobre los datos. Para evaluar esto, se generaron tres PCA plots según las variables estudiadas.

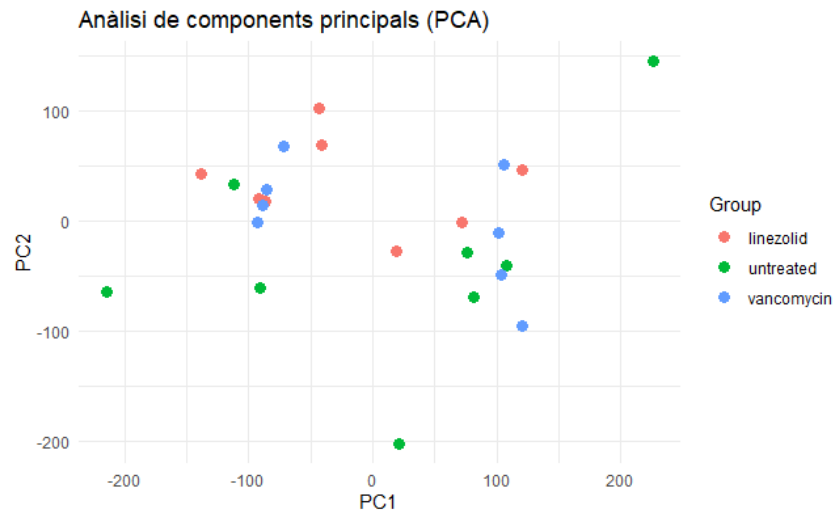


Figura 7: PCA según el grupo **agent**. Cada punto representa una muestra coloreada según el tratamiento.

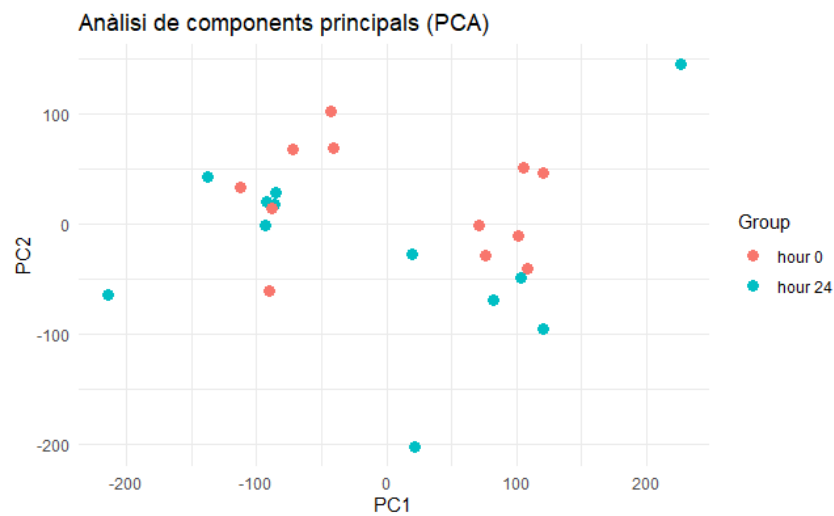


Figura 8: PCA según la variable **time**. Cada punto representa una muestra coloreada según el tiempo de muestreo.

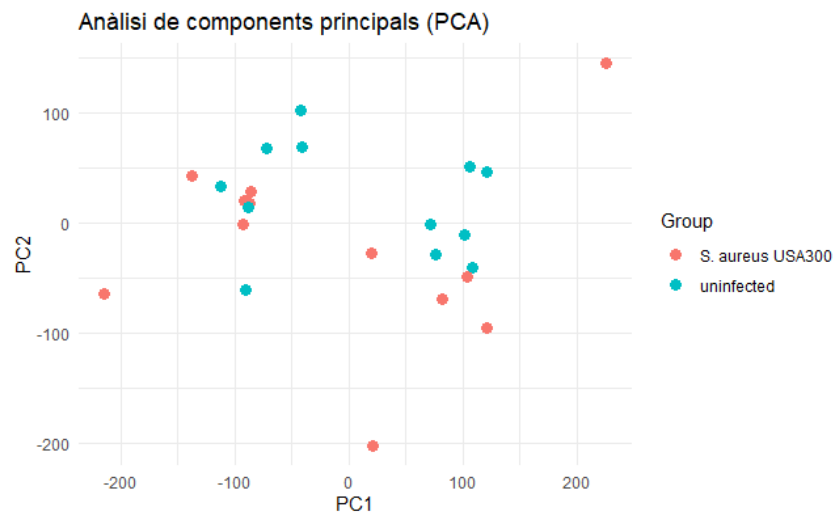


Figura 9: PCA según la variable `infection`. Cada punto representa una muestra coloreada según la condición de infección.

No se observa un agrupamiento claro debido a alguna de las variables estudiadas, lo que sugiere la existencia de una fuente de variabilidad no contemplada que genera la separación de las muestras en dos bloques. Esto podría deberse a interacciones complejas entre variables o a efectos no lineales que el PCA lineal no logra capturar.

Análisis de Variabilidad de Factores de *Batch* (PVCA)

Se evaluó la variabilidad de las muestras considerando los posibles factores de *batch*, en este caso las variables `agent`, `infection` y `time`, mediante la función `pvcaBatchAssess` del paquete `pvca`. Se estableció un umbral del 50% para la proporción de variabilidad explicada.

```
library(pvca)
pData(normalized_data) <- Targets
pct_threshold <- 0.5
batch.factors <- c("agent", "infection", "time")
pvcaObj <- pvcaBatchAssess(normalized_data, batch.factors, pct_threshold)
```

Los resultados se visualizaron en un gráfico de barras, mostrando la proporción de variabilidad explicada por cada factor o interacción de factores:

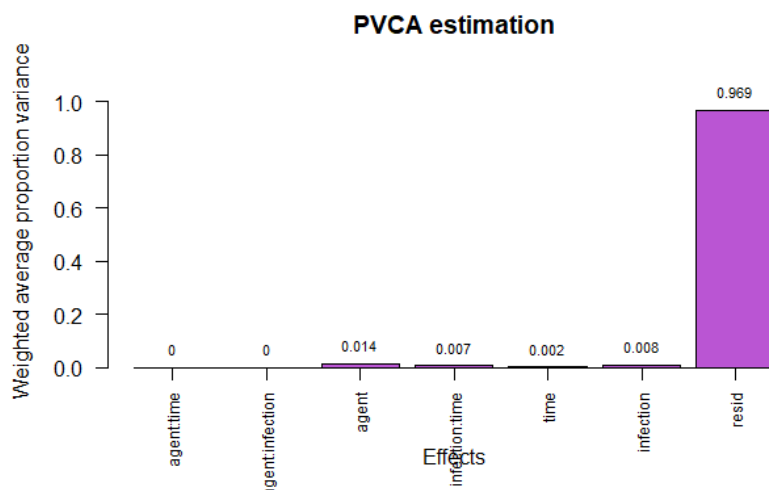


Figura 10: Proporción de variabilidad explicada por los factores **agent**, **infection** y **time** según PVCA.

Los resultados indican que los factores considerados no capturan una proporción significativa de la variabilidad observada en los datos, sugiriendo la presencia de fuentes de variabilidad no identificadas o no incluidas en el análisis.

2.3. Filtro de los Datos

El filtrado de sondas es un paso crítico en el análisis de microarrays, aunque su aplicación es objeto de debate. En este caso, se decidió conservar únicamente el 10 % de las sondas con mayor variabilidad, eliminando aquellas con variación baja o información redundante.

Se utilizó la función `nsFilter` del paquete `genefilter` con los siguientes criterios:

- Solo se conservaron sondas con un *Entrez Gene ID* válido (`require.entrez = TRUE`).
- Se eliminaron duplicados de *Entrez Gene ID* (`remove.dupEntrez = TRUE`).
- Se seleccionaron sondas con alta variabilidad, evaluada mediante el *Interquartile Range* (IQR), considerando únicamente aquellas con `var.cutoff = 0.9`.
- Se excluyeron las sondas cuyo identificador comenzaba con **AFFX**, correspondientes a controles internos o información adicional.
- Se aplicó un filtrado por cuantiles para eliminar genes con distribuciones muy sesgadas o valores extremos que pudieran afectar el análisis.

El resultado de `nsFilter` es una lista que contiene los datos filtrados y el nuevo `ExpressionSet`. El `ExpressionSet` resultante se almacena en `filtered_data`:

```
ExpressionSet (storageMode: lockedEnvironment)
assayData: 2048 features, 24 samples
  element names: exprs
protocolData
```

```

sampleNames: GSM944831.CEL GSM944833.CEL ... GSM944865.CEL (24
  total)
varLabels: ScanDate
varMetadata: labelDescription
phenoData
  sampleNames: GSM944850 GSM944843 ... GSM944862 (24 total)
  varLabels: sample infection ... Group (5 total)
  varMetadata: labelDescription
featureData: none
experimentData: use 'experimentData(object)'
Annotation: mouse4302

```

Este procedimiento permite centrar el análisis en las sondas más informativas, eliminando ruido y datos irrelevantes para los análisis posteriores.

2.4. Construcción de la Matriz de Diseño y de Contraste

Para definir la matriz de diseño se utilizó el paquete `limma`. Se creó una nueva variable `Group` en los metadatos (`pData`) que combina las variables `infection` y `agent` separadas por un guion bajo (`_`). Se eliminaron los puntos y se reemplazaron los espacios por guiones bajos para evitar conflictos con los nombres de las variables.

```

library(limma)
pData(filtered_data)$Group <- paste(pData(filtered_data)$infection,
                                   pData(filtered_data)$agent,
                                   sep = "_")
pData(filtered_data)$Group <- gsub("\\.", "", pData(filtered_data)$Group)
pData(filtered_data)$Group <- gsub(" ", "_", pData(filtered_data)$Group)

```

A continuación, se creó el objeto `groups` con la información de la variable `Group`, y se diseñó un modelo lineal sin intercepto, especificando los niveles de los grupos como nombres de columna de la matriz de diseño.

```

groups <- pData(filtered_data)$Group
design <- model.matrix(~0 + factor(groups))
colnames(design) <- levels(factor(groups))

```

Se verificó la distribución de muestras por grupo experimental y la correcta construcción de la matriz de diseño.

Posteriormente, se construyó la matriz de contrastes mediante la función `makeContrasts`, definiendo tres comparaciones: uninfected vs *S. aureus* para cada condición de infección. Es fundamental que los nombres de los contrastes coincidan con los niveles de los grupos definidos en la matriz de diseño.

```

contrast_matrix <- makeContrasts(
  Grup1_vs_Grup2 = uninfected_untreated - S_aureus_USA300_untreated,
  Grup1_vs_Grup3 = uninfected_linezolid - S_aureus_USA300_linezolid,
  Grup2_vs_Grup3 = uninfected_vancomycin - S_aureus_USA300_vancomycin,
  levels = design
)

```

La matriz de contrastes resultante es la siguiente:

Levels	G1_vs_G2	G1_vs_G3	G2_vs_G3
S_aureus_USA300_linezolid	0	-1	0
S_aureus_USA300_untreated	-1	0	0
S_aureus_USA300_vancomycin	0	0	-1
uninfected_linezolid	0	1	0
uninfected_untreated	1	0	0
uninfected_vancomycin	0	0	1

Cuadro 1: Matriz de contrastes para los diferentes grupos experimentales.

Esta matriz permitirá realizar los análisis de expresión diferencial entre los grupos definidos.

2.5. Obtención de los Genes Diferencialmente Expresados

El análisis de expresión diferencial se realizó aplicando un modelo lineal sobre los datos filtrados (`filtered_data`) utilizando la matriz de diseño previamente construida. A continuación, se aplicaron los contrastes definidos y se efectuó una estimación bayesiana para mejorar la estabilidad de las varianzas.

```
fit <- lmFit(exprs(filtered_data), design)      # Ajuste del modelo lineal
fit2 <- contrasts.fit(fit, contrast_matrix)     # Aplicar contrastes
fit2 <- eBayes(fit2)                           # Estimación Bayesiana
```

Los resultados de los contrastes se almacenan en `fit2`. Para extraer los genes diferencialmente expresados se utilizó la función `topTable()`, especificando el coeficiente de interés y el método de ajuste de p-valor (`fdr`):

```
Grup1_vs_Grup2 <- topTable(fit2, number=nrow(fit2), coef="Grup1_vs_Grup2",
                           adjust="fdr")
Grup1_vs_Grup3 <- topTable(fit2, number=nrow(fit2), coef="Grup1_vs_Grup3",
                           adjust="fdr")
Grup2_vs_Grup3 <- topTable(fit2, number=nrow(fit2), coef="Grup2_vs_Grup3",
                           adjust="fdr")
```

En este análisis, ningún contraste arrojó un p-valor ajustado significativo, probablemente debido al efecto batch y a la variabilidad no explicada identificada previamente. Para continuar con el análisis, se consideraron los p-valores sin ajustar, reconociendo que estos no corrigen el error de tipo I asociado a las pruebas múltiples:

```
g1_vs_g2 <- Grup1_vs_Grup2[Grup1_vs_Grup2$P.Value < 0.05, ]
g1_vs_g3 <- Grup1_vs_Grup3[Grup1_vs_Grup3$P.Value < 0.05, ]
g2_vs_g3 <- Grup2_vs_Grup3[Grup2_vs_Grup3$P.Value < 0.05, ]
```

Estos subconjuntos contienen los genes con p-valor menor a 0.05, y constituyen la base para posteriores análisis funcionales o de visualización.

Cuadro 2: Genes diferencialmente expresados para los tres contrastes principales. Se muestran los 6 primeros genes de cada contraste.

ProbeID	logFC	AveExpr	t	P.Value	adj.P.Val	B
Grup1 vs Grup2						
1427084_a_at	0.907	7.234	2.163	0.0413	0.9844	-4.526
1425005_at	0.805	6.121	2.125	0.0447	0.9844	-4.529
1460197_a_at	-1.908	6.686	-2.117	0.0454	0.9844	-4.529
1451536_at	0.862	9.250	2.099	0.0471	0.9844	-4.530
1418216_at	-0.868	6.629	-2.027	0.0545	0.9844	-4.535
1449235_at	0.908	5.738	1.988	0.0589	0.9844	-4.538
Grup1 vs Grup3						
1450699_at	1.437	7.438	3.819	0.000892	0.8108	-4.405
1436766_at	-1.202	6.300	-3.056	0.00564	0.8108	-4.460
1429808_at	1.054	6.407	2.815	0.00988	0.8108	-4.478
1428900_s_at	0.993	6.247	2.671	0.0137	0.8108	-4.489
1450919_at	0.972	12.968	2.607	0.0158	0.8108	-4.493
1425814_a_at	1.404	8.631	2.584	0.0167	0.8108	-4.495
Grup2 vs Grup3						
1427161_at	1.518	7.305	3.427	0.00232	0.9999	-4.433
1417133_at	-1.506	6.103	-3.051	0.00570	0.9999	-4.460
1426817_at	0.990	10.462	2.995	0.00651	0.9999	-4.465
1453220_at	1.142	6.848	2.912	0.00790	0.9999	-4.471
1452661_at	1.283	12.367	2.912	0.00790	0.9999	-4.471
1433623_at	1.090	6.659	2.884	0.00843	0.9999	-4.473

2.6. Anotación de los Genes

El `ExpressionSet` utilizado está basado en el Affymetrix Mouse Genome 430 2.0 Array, como se indica en `Annotation=mouse4302`. En las tablas obtenidas mediante `topTable`, las filas se identifican con los identificadores de Affymetrix, accesibles mediante `rownames()`. Adicionalmente, el proyecto GEO proporciona un archivo de anotaciones (`GPL1261.annot`) que incluye información sobre los genes, como el símbolo (`Symbol`), el identificador `Entrez` y detalles sobre la actividad biológica.

Para realizar el mapeo de sondas a identificadores `ENTREZID` se utilizó el paquete de anotaciones `mouse4302.db` junto con la función `mapIds`, considerando que las claves originales están en formato `PROBEID`:

```
library(mouse4302.db)

# Mapeado del primer contraste
mapped_ids12 <- mapIds(
  mouse4302.db,
  keys = rownames(g1_vs_g2),
  column = "ENTREZID",
  keytype = "PROBEID",
  multiVals = "first"
)
```

Cuadro 3: Genes diferencialmente expresados con $P.\text{Value} < 0.05$ para los tres contrastes principales. Se muestran los 6 primeros genes de cada contraste.

ProbeID	logFC	AveExpr	t	P.Value	adj.P.Val	B
g1 vs g2						
1427084_a_at	0.907	7.234	2.163	0.04132	0.98438	-4.526
1425005_at	0.805	6.121	2.125	0.04466	0.98438	-4.529
1460197_a_at	-1.908	6.686	-2.117	0.04541	0.98438	-4.529
1451536_at	0.862	9.250	2.099	0.04707	0.98438	-4.530
g1 vs g3						
1450699_at	1.437	7.438	3.819	0.000892	0.81084	-4.405
1436766_at	-1.202	6.300	-3.056	0.005642	0.81084	-4.460
1429808_at	1.054	6.407	2.815	0.009882	0.81084	-4.478
1428900_s_at	0.993	6.247	2.671	0.01370	0.81084	-4.489
1450919_at	0.972	12.968	2.607	0.01582	0.81084	-4.493
1425814_a_at	1.404	8.631	2.584	0.01665	0.81084	-4.495
g2 vs g3						
1427161_at	1.518	7.305	3.427	0.002324	0.99989	-4.433
1417133_at	-1.506	6.103	-3.051	0.005702	0.99989	-4.460
1426817_at	0.990	10.462	2.995	0.006510	0.99989	-4.465
1453220_at	1.142	6.848	2.912	0.007897	0.99989	-4.471
1452661_at	1.283	12.367	2.912	0.007904	0.99989	-4.471
1433623_at	1.090	6.659	2.884	0.008432	0.99989	-4.473

```
# Mapeado del segundo contraste
```

```
mapped_ids13 <- mapIds(
  mouse4302.db,
  keys = rownames(g1_vs_g3),
  column = "ENTREZID",
  keytype = "PROBEID",
  multiVals = "first"
)
```

```
# Mapeado del tercer contraste
```

```
mapped_ids23 <- mapIds(
  mouse4302.db,
  keys = rownames(g2_vs_g3),
  column = "ENTREZID",
  keytype = "PROBEID",
  multiVals = "first"
)
```

Posteriormente, los identificadores ENTREZID se añadieron como una nueva columna en los resultados de cada contraste:

```
g1_vs_g2$ENTREZID <- mapped_ids12
g1_vs_g3$ENTREZID <- mapped_ids13
```



```
g2_vs_g3$ENTREZID <- mapped_ids23
```

De este modo, la lista de genes significativos incluye ahora la columna `ENTREZID`, proporcionando el identificador estándar de cada gen para análisis posteriores.

2.7. Análisis de la Significación Biológica

Cada lista de genes contiene aquellos que se han expresado diferencialmente en cada contraste. El análisis de la significación biológica permite identificar genes que podrían tener un impacto funcional relevante o que podrían estar implicados en procesos biológicos clave.

Se realizó un análisis de enriquecimiento de *Gene Ontology* utilizando el paquete `clusterProfiler` y la función `enrichGO`. En este análisis se especificaron los identificadores de los genes (`gene`), la base de anotaciones utilizada (`OrgDb`), el tipo de identificador (`keyType`) y la ontología de interés (`ont`):

```
library(clusterProfiler)

enrich_result12 <- enrichGO(
  gene = g1_vs_g2$ENTREZID,
  OrgDb = mouse4302.db,
  keyType = "ENTREZID",
  ont = "BP",
  pAdjustMethod = "BH"
)

enrich_result13 <- enrichGO(
  gene = g1_vs_g3$ENTREZID,
  OrgDb = mouse4302.db,
  keyType = "ENTREZID",
  ont = "BP",
  pAdjustMethod = "BH"
)

enrich_result23 <- enrichGO(
  gene = g2_vs_g3$ENTREZID,
  OrgDb = mouse4302.db,
  keyType = "ENTREZID",
  ont = "BP",
  pAdjustMethod = "BH"
)
```

Para visualizar los resultados del análisis de GO, se generaron diagramas de puntos (`dotplot`) para cada contraste:

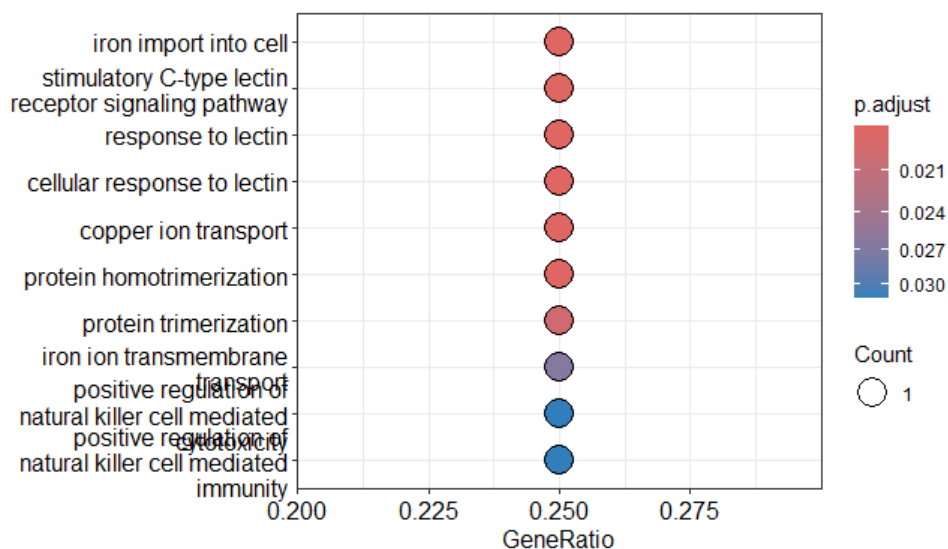


Figura 11: Enriquecimiento GO: g1 vs g2 (untreated). Este gráfico muestra los términos GO más significativos con su respectiva proporción de genes (GeneRatio) y el p-valor ajustado.

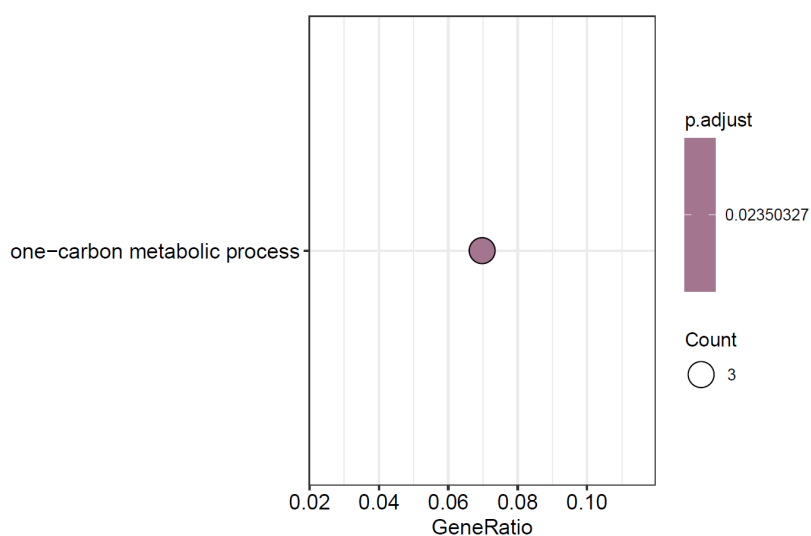


Figura 12: Enriquecimiento GO: g1 vs g3 (linezolid). Este gráfico muestra los términos GO más significativos con su respectiva proporción de genes (GeneRatio) y el p-valor ajustado.

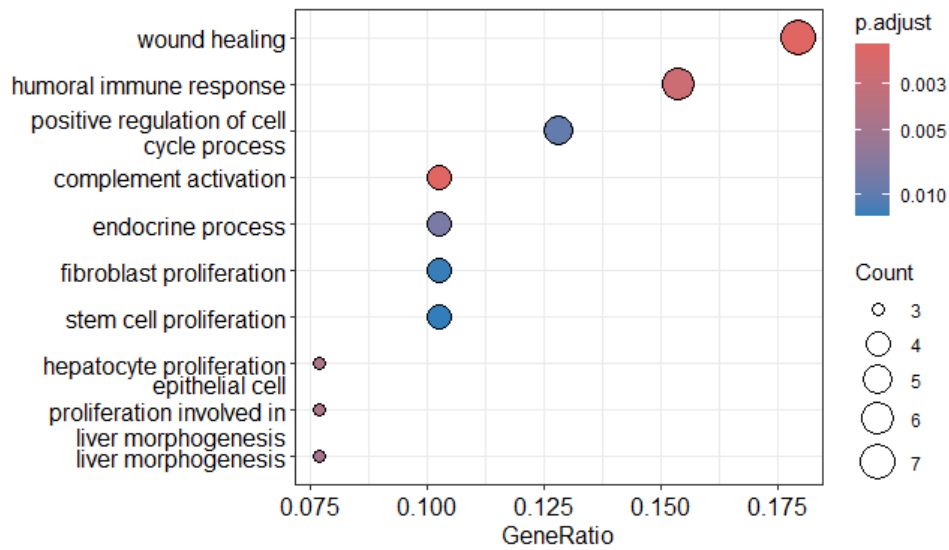


Figura 13: Enriquecimiento GO: g2 vs g3 (vancomycin). Este gráfico muestra los términos GO más significativos con su respectiva proporción de genes (GeneRatio) y el p-valor ajustado.

Estos gráficos permiten identificar los términos de *Gene Ontology* más enriquecidos en cada lista de genes, facilitando la interpretación biológica de los resultados.

3. Resultados

Los tres gráficos generados representan los resultados de enriquecimiento de *Gene Ontology* (GO) y reflejan diferencias entre las condiciones experimentales. En cada gráfico, el *GeneRatio* indica la proporción de genes de la lista del contraste asociados con cada término GO.

En el primer gráfico (untreated) se observan múltiples términos GO relacionados con procesos biológicos como respuesta inmune, cicatrización y desarrollo hepático. Todos los procesos presentan un *GeneRatio* similar, aunque algunos muestran un p-valor ajustado más significativo que otros. Esto indica que los genes diferencialmente expresados se deben principalmente a la infección con *S. aureus*, dado que no existe otro tratamiento en esta condición.

En el segundo gráfico (linezolid) únicamente aparece un término GO, correspondiente al *One-carbon metabolic process*, con un *GeneRatio* bajo y un p-valor ajustado significativo. Este contraste evalúa los efectos del tratamiento con linezolid en presencia de la infección bacteriana, sugiriendo un efecto específico del antibiótico sobre el metabolismo del carbono.

En el tercer gráfico (vancomycin) se destacan procesos relacionados con transporte de hierro, respuesta de lectinas y transporte iónico, entre otros términos con *GeneRatio* elevado. Estos términos reflejan mecanismos específicos de acción del antibiótico o respuestas celulares frente a la infección, mostrando los efectos particulares de la combinación de infección y tratamiento con vancomicina.